DATA 404

Assignment 2: Large-Scale Database Implementation

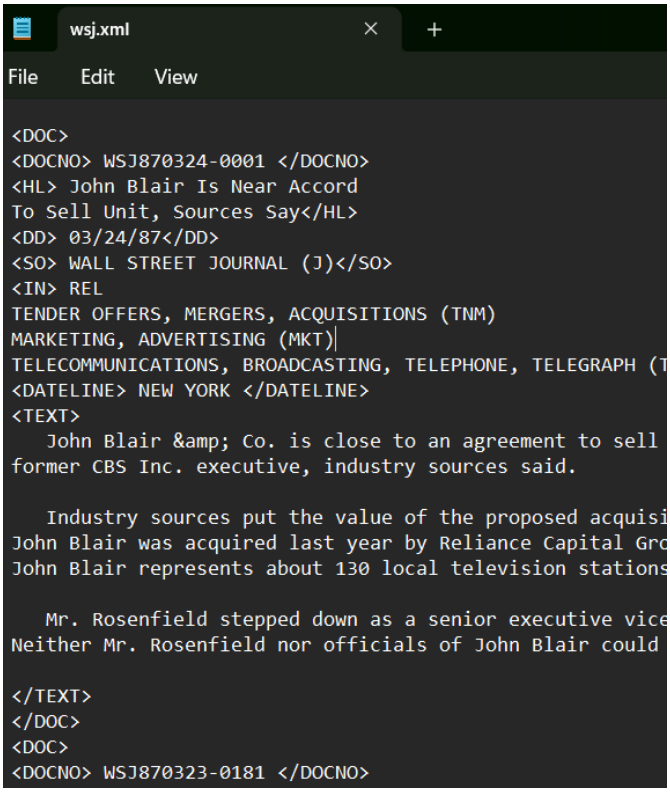
Name : Siddharth Barjatya | Student ID :

Contents

1 Dataset Overview 2
1.1 Dataset Description 2
1.2 Dataset Characteristics 3
1.3 Structure and Complexity 3
1.4 Technical Considerations 4
2 DBMS Selection and Justification 5
2.1 Chosen DBMS 5
2.2 Justification & Alternatives Considered 6
2.3 Analysis Use Cases 7
3 Database Design and Implementation 7
3.1 Schema / Data Model 8
3.2 Design Decisions and Rationale 9
3.3 Graph Analysis Use Cases 11
3.4 Relational Dashboard 14
4 Appendix - Data Import 15
4.1 Import Method and Tools 15
4.2 Schema Code & Data Import Script 15
4.3 Data Preprocessing Script 17

1 Dataset Overview

The dataset used in this assignment is a raw XML-based news feed from the Wall Street Journal (wsj.com), stored in a single file named wsj.xml. The data collection is a standard set of full-text news articles used for information retrieval experiments. The original data format is a custom XML markup scheme, loosely resembling an RSS (Really Simple Syndication) feed. RSS is a standardised XML-based format widely used to aggregate and distribute news content, allowing consumers to track updates from multiple sources without visiting each site individually. However, because this collection predates the formalisation of the RSS standard, the file is not well-formed XML and does not parse correctly with a standard XML parser, making it unsuitable for direct ingestion via conventional XML tooling and requiring a custom parsing pipeline. The Wall Street Journal dataset is a collection of documents marked up in XML that look roughly like this:



1.1 Dataset Description

The collections include articles from the Wall Street Journal, specifically years 1990, 1991, and 1992. It is a collection of over 170,000 articles published between April 1990 and March 1992, with an uncompressed size of 508 MB (533,196,800 bytes) on disk. Each article constitutes one record, giving 173,252 records in total. Each article is delimited by <DOC>...</DOC> tags and contains some or all of the following fields (<DOCNO> <DOCID> <DATE> <AUTHOR> <DATELINE> <AN> <CO> <DD> <G> <GV> <HL> <IN> <LP> <MS> <NS> <RE> <SO> <ST> <TEXT>) :

Field	Description
<DOCNO>	Unique document identifier (e.g., WSJ870324-0001)
<HL>	Article headline
<DD>	Publication date in MM/DD/YY format

<SO>	Source publication
<IN>	Industry/topic codes and region codes (multi-valued)
<DATELINE>	Geographic origin of the article
<TEXT>	Full article body text

1.2 Dataset Characteristics

Source	The dataset was sourced from the standard TREC (Text REtrieval Conference) document corpus, which has been widely used in information retrieval research.
Total size (uncompressed)	508 MB (53,31,96,800 bytes) containing more than 170K articles
Original format	XML (Raw, unprocessed & cannot be parsed with standard parsers)
Licence	No sensitive personal information is present in the dataset, and it is freely available for academic use.

1.3 Structure and Complexity

tag	open_count	close_count	doc_count	count_equal	count_within_range
ROOT	0	0	173252	TRUE	TRUE
DOC	173252	173252	173252	TRUE	TRUE
DOCNO	173252	173252	173252	TRUE	TRUE
HL	173248	173248	173252	TRUE	TRUE
DD	69321	69321	173252	TRUE	TRUE
SO	173227	173227	173252	TRUE	TRUE
IN	143952	143952	173252	TRUE	TRUE
DATELINE	60607	60607	173252	TRUE	TRUE
TEXT	173252	173252	173252	TRUE	TRUE
CO	95536	95536	173252	TRUE	TRUE
AUTHOR	14371	14371	173252	TRUE	TRUE
G	6145	6145	173252	TRUE	TRUE
AN	12380	12380	173252	TRUE	TRUE
GV	16929	16929	173252	TRUE	TRUE
DOCID	74520	74520	173252	TRUE	TRUE
DATE	74519	74519	173252	TRUE	TRUE
LP	74486	74486	173252	TRUE	TRUE
MS	32442	32442	173252	TRUE	TRUE
RE	38393	38393	173252	TRUE	TRUE
NS	30941	30941	173252	TRUE	TRUE
ST	330	330	173252	TRUE	TRUE

Below is a list of few points that together made it a reasonable case for why the dataset is non-trivial to load and why schema design decisions (e.g., how to handle multi-valued <IN>, whether to normalise topic codes, how to index free text) made it quite interesting.

- **Semi-structured XML format requiring parsing**

  The data is not a flat table as each document is wrapped in a custom XML-like schema with distinct named tags (<DOCNO>, <HL>, <DD>, <SO>, <IN>, <DATELINE>, <TEXT>). Any import pipeline needs to parse this tag structure rather than simply reading rows.

- **Multi-valued <IN> field**

The industry/topic codes field frequently contains multiple values on separate lines. For example, the first document has four distinct values under <IN> (a region code REL, plus three verbose category strings). This is a one-to-many relationship packed into a single field, which requires either normalisation into a separate table/collection or careful string splitting.

- **Mixed encoding: code vs. free text in <IN>**

  Within <IN>, some values are short opaque codes (FREST, EUROP, CANDA, REL) while others are full descriptive strings (TENDER OFFERS, MERGERS, ACQUISITIONS (TNM)). These appear to follow a pattern of [region code] followed by zero or more [topic string (topic_code)] lines, but this is implicit rather than schema-enforced.

- **HTML entity encoding in body text**

  The <TEXT> field contains HTML entities such as & (in "John Blair & Co."), which need to be decoded during ingestion and are not consistent across documents; most articles contain plain text with no entities.

- **Highly variable document length**

  Text length varies enormously: some documents are a single sentence (WSJ870323-0179, WSJ870323-0180), while others are several paragraphs. This creates uneven storage and indexing characteristics, particularly relevant when loading into a document or full-text search DBMS.

- **Implicit date format**

  Dates are stored as MM/DD/YY strings (e.g., 03/24/87) rather than a standard ISO 8601 format, requiring transformation before any date-based querying or sorting.

- **Absent fields**

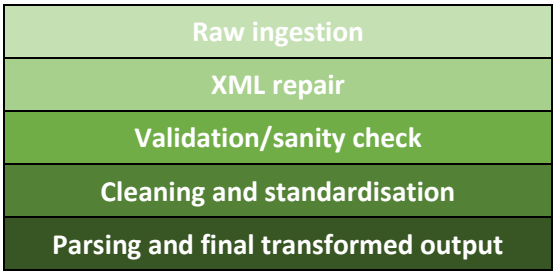
  Not all fields are present in every document, for instance, some articles may lack a <DATELINE> or have a minimal <IN> block. The schema has no enforced required fields, so any implementation needs to handle nulls gracefully.

- **No explicit record delimiter or document count**

  Documents are concatenated sequentially with no wrapping root element or record count, meaning the file is not valid XML at the corpus level as only individual <DOC> blocks are well-formed. This affects how you can stream or chunk the data during import.

1.4 Technical Considerations

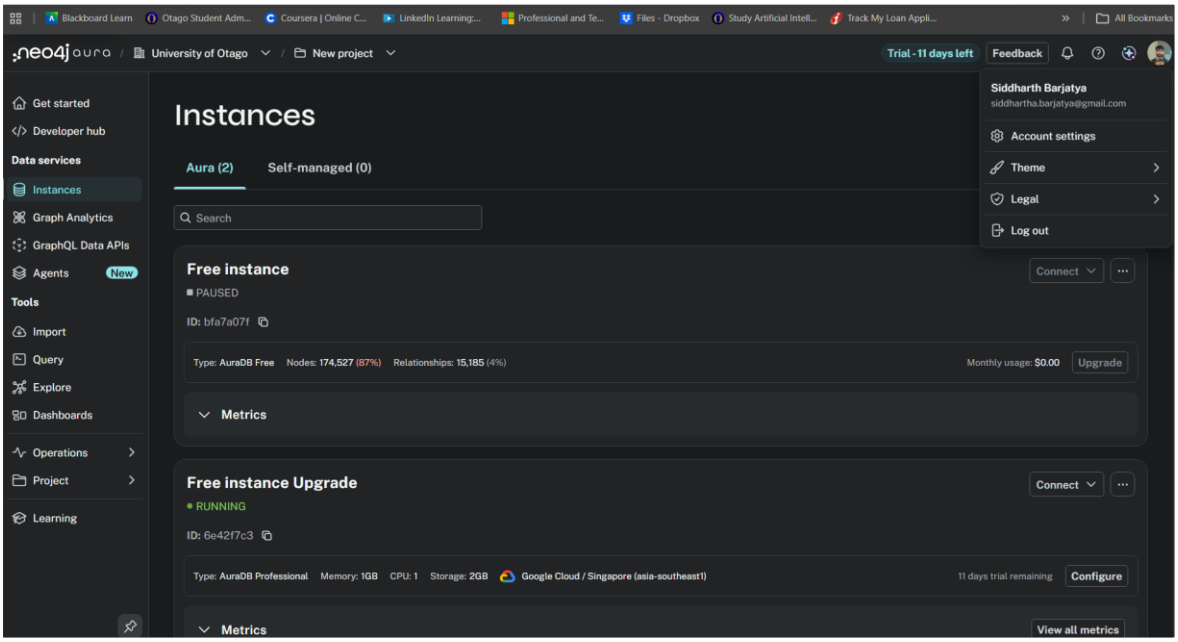
A key technical consideration was that the corpus file was not a valid XML document at the top level individual <DOC> blocks were self-contained, but the file had no root element wrapping them, meaning the entire file cannot be parsed as a single XML tree. This, combined with the multi-valued and mixed-format <AUTHOR> <IN> <MS> <NS> <RE> fields and the presence of HTML entities such as & in body text, meant that the dataset required a bespoke ingestion pipeline rather than off-the-shelf XML or RSS tooling. The data pipeline broadly looked like :

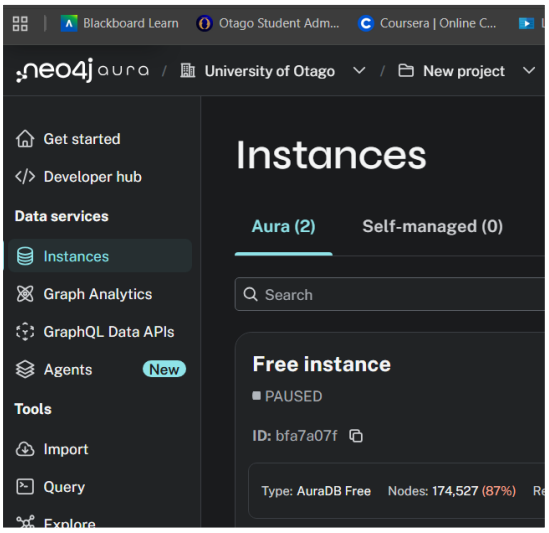


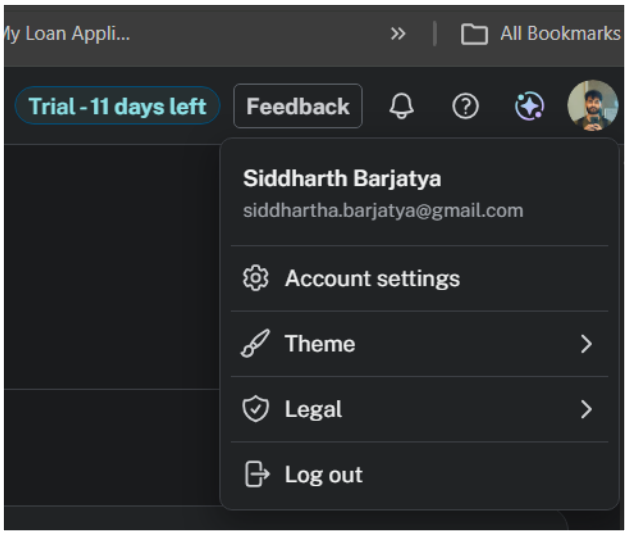
2 DBMS Selection and Justification

2.1 Chosen DBMS

The chosen DBMS for this implementation is Neo4j (Aura Console (console.neo4j.io)), a native graph database that stores data as a property graph, essentially a network of nodes connected by typed, directed relationships, each of which can carry its own properties. Unlike relational systems where relationships are inferred at query time through joins, Neo4j encodes relationships as first-class structural elements, making traversal across connected entities a core operation rather than a derived one. Neo4j is queried using Cypher, a declarative pattern-matching language designed specifically for graph traversal. Although Neo4j was not covered in DATA 404, it works quite similar to Arcade DB i.e. data is represented as nodes (analogous to entities) and edges (relationships between entities), both of which can be labelled and carry arbitrary key-value properties. There is no enforced schema, though constraints and indexes can be applied to specific node labels and property keys.







2.2 Justification & Alternatives Considered

The central structural characteristic of the WSJ dataset is the repeated occurrence of multi-valued fields attached to a single document identifier. Fields such as <IN>, <RE>, <CO>, <MS>, and <NS> contain lists of codes or descriptors - industry categories, geographic regions, company identifiers, market sectors, and news subjects respectively, while <AUTHOR> may reference one or more named individuals.

In a relational model, each of these fields would require a separate normalised junction table (e.g. document_industry, document_region, document_company), resulting in a schema of at least seven or eight tables with foreign key relationships between them. Queries spanning multiple dimensions for instance, retrieving all articles about a given company in a given region written by a particular author would require multi-way joins across several of these tables, incurring significant overhead at the scale of 170,000 documents.

A graph model eliminates this overhead structurally. Each unique value from a multi-valued field becomes a node with its own label (e.g. :Industry, :Region, :Company, :Author), and each document-value association becomes a labelled relationship (e.g. (:Article)-[:TAGGED_WITH]->(:Industry)). Queries that traverse these relationships finding all articles connecting a given author to a given industry sector, or identifying which regions co-occur most frequently with a given company are expressed naturally as Cypher path patterns and executed via index-free adjacency, where each node holds direct pointers to its neighbours rather than requiring a table scan or join resolution.

The dataset's vocabulary fields also exhibit implicit network structure that a graph database is uniquely positioned to expose. Two documents sharing multiple <IN> or <RE> codes are thematically related as two authors who frequently co-appear on articles about the same companies form a latent collaboration network. These relationships are invisible in a flat or relational representation but emerge directly from graph traversal. A graph model therefore not only stores the data faithfully but makes its inherent connectivity queryable without additional transformation.

Document-oriented alternatives such as MongoDB were considered. While MongoDB handles multi-valued fields naturally as arrays within a document, it does not natively represent or traverse cross-document relationships, hence finding all documents linked to a given <IN> code would require either a full collection scan or a manual index lookup, and multi-hop queries (e.g. articles -to- shared authors -to- other articles) have no native path-traversal support.

DBMS	**Type**	**Reason not chosen**
PostgreSQL	Relational (SQL)	Seven multi-valued fields require seven junction tables; multi-hop relationship queries across Author, Company, GovtBody, and Region simultaneously become expensive multi-way joins at 170K document scale
MongoDB	Document store	Multi-valued fields are naturally stored as arrays within a document, but cross-document relationship traversal is not a native operation; co-occurrence and network queries across node types require application-level logic with no path-traversal primitive

2.3 Analysis Use Cases

While the scope of this assignment is implementation rather than analysis, a set of plausible analytical queries informed both the choice of DBMS and the design of the graph schema:

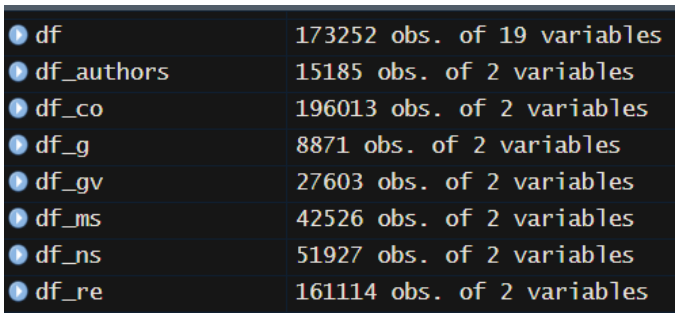
- Co-occurrence queries : which industry codes (<IN>) or geographic regions (<RE>) appear together most frequently across articles? In a graph, this is a two-hop traversal: (:Industry)<-[:TAGGED_WITH]-(:Article)-[:TAGGED_WITH]->(:Industry).
- Author–topic profiling : which industry sectors does a given author write about, and how does this change over time? This traversal spans :Author, :Article, and :Industry nodes with date filtering on article properties.
- Company network analysis : which companies (<CO>) are mentioned together in the same articles, and do certain company clusters correspond to particular market sectors (<MS>)? This is a bipartite graph query naturally suited to Cypher's pattern matching.
- Editorial coverage analysis : how does the volume of articles per region (<RE>) or subject code (<NS>) vary across the two-year publication window?

Each of these queries is expressible as a short Cypher pattern and would require complex multi-join SQL or application-level post-processing in a relational system.

3 Database Design and Implementation

As an end result of the data preprocessing step, the following files were produced as CSVs.

R pipeline Dataframes



Node files

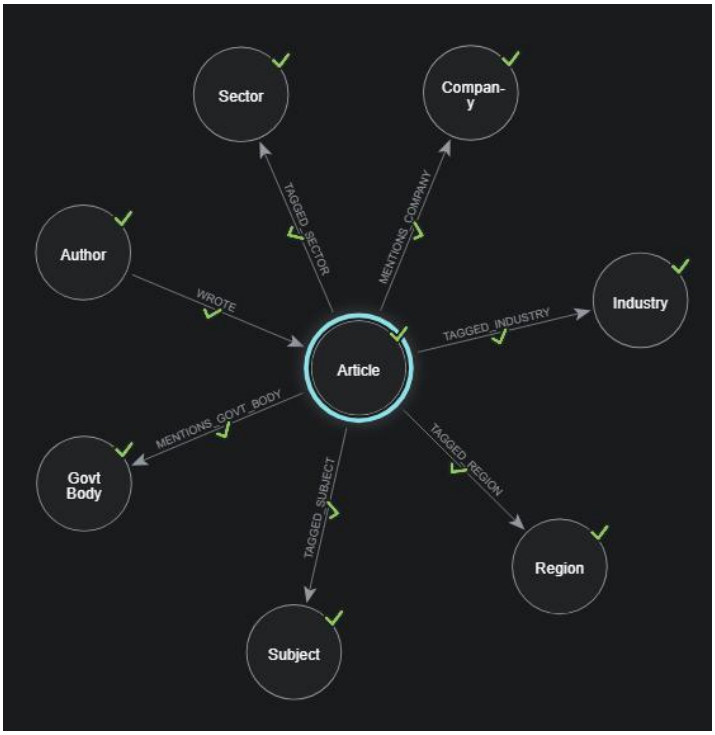
File	Node Label	Key Property
articles.csv	Article	DOCNO
authors.csv	Author	name
companies.csv	Company	code
g_nodes.csv	GTag	tag
govt_bodies.csv	GovtBody	name
regions.csv	Region	name
sectors.csv	Sector	name
subjects.csv	Subject	name

Edge files

File	Relationship	From → To
wrote_edges.csv	WROTE	Author → Article
co_edges.csv	MENTIONS_COMPANY	Article → Company
g_edges.csv	TAGGED_INDUSTRY	Article → GTag
gv_edges.csv	MENTIONS_GOVT_BODY	Article → GovtBody
re_edges.csv	TAGGED_REGION	Article → Region
ms_edges.csv	TAGGED_SECTOR	Article → Sector
ns_edges.csv	TAGGED_SUBJECT	Article → Subject

3.1 Schema / Data Model

The defining structural characteristic of the WSJ dataset is that a single article is identified by its DOCNO and is simultaneously associated with multiple values across several categorical metadata fields. Specifically, the fields RE (geographic regions), MS (market sectors), NS (news subjects), CO (company identifiers), GV (government bodies), and AUTHOR are all multi-valued, with each article carrying anywhere from zero to a dozen values per field. This is not incidental to the dataset as it is the primary way the corpus encodes what an article is about, who wrote it, and which institutions it concerns.



In a relational model, faithfully representing this structure requires a separate normalised junction table for each of these seven fields, yielding a schema of at least seven tables connected by foreign keys. A query asking which industry topics a given author covers across articles mentioning a specific government body would require a five-way join across articles, wrote_edges, authors, co_edges, and gv_edges at 173252 documents and hundreds of thousands of edge rows, this becomes expensive both to write and to execute. A graph model resolves this structurally. Each unique value from a multi-

valued field becomes a dedicated node with its own label, and each article-to-value association becomes a typed relationship. The resulting schema for this implementation is:

Node label	Source field	Description
Article	DOCNO, DATE	One node per article
Author	AUTHOR	Individual byline names
Company	CO	Company identifiers
GovtBody	GV	Government bodies and agencies
Region	RE	Geographic region codes
Sector	MS	Market sector classifications
Subject	NS	News subject codes

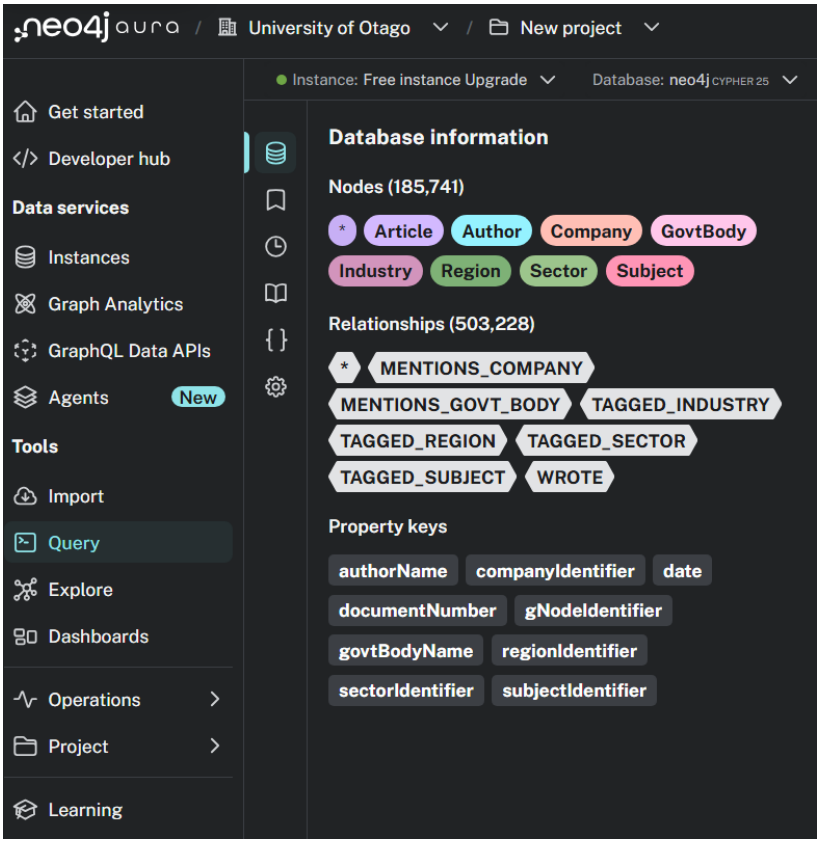
3.2 Design Decisions and Rationale

These nodes are connected by the following relationships, all originating from or terminating at an Article node:

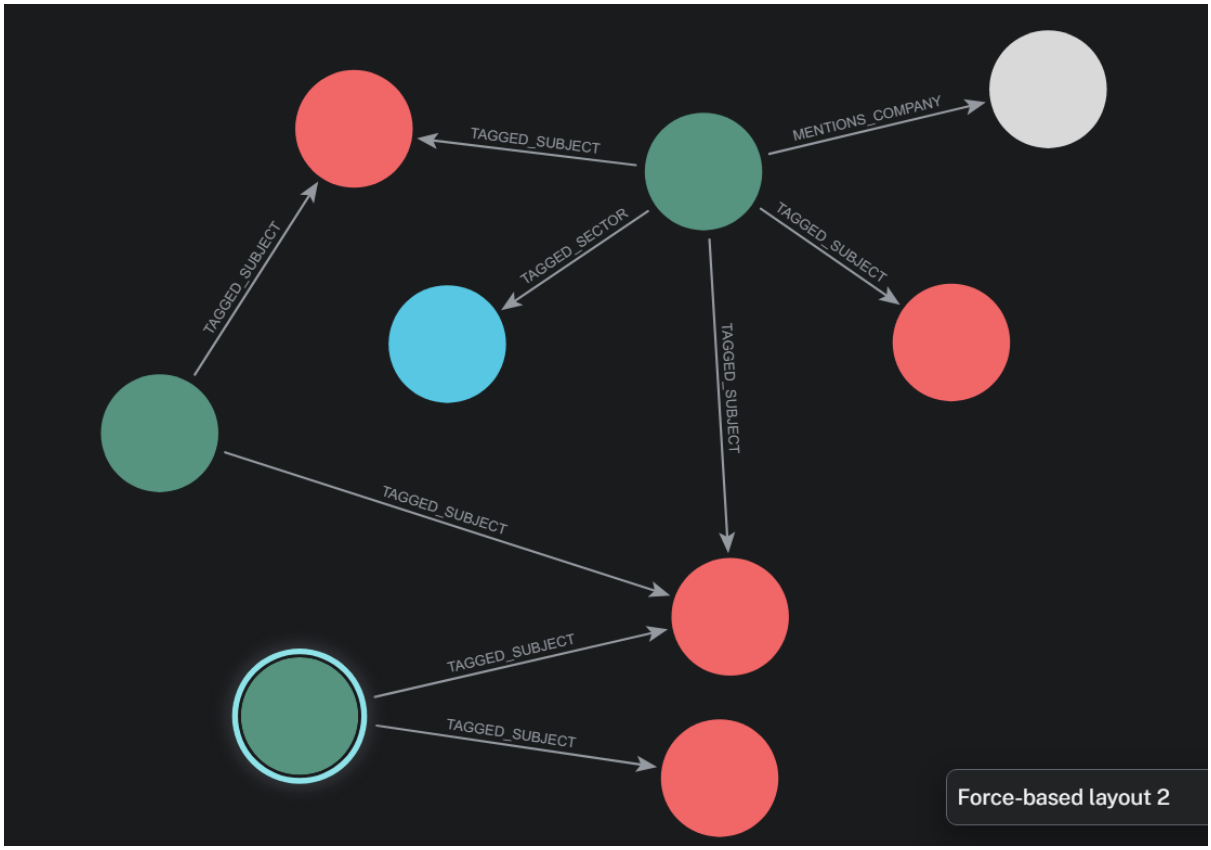
Relationship	Pattern
:WROTE	(Author)-[:WROTE]->(Article)
:MENTIONS_COMPANY	(Article)-[:MENTIONS_COMPANY]->(Company)
:MENTIONS_GOVT_BODY	(Article)-[:MENTIONS_GOVT_BODY]->(GovtBody)
:TAGGED_REGION	(Article)-[:TAGGED_REGION]->(Region)
:TAGGED_SECTOR	(Article)-[:TAGGED_SECTOR]->(Sector)
:TAGGED_SUBJECT	(Article)-[:TAGGED_SUBJECT]->(Subject)
:TAGGED_INDUSTRY	(Article)-[:TAGGED_INDUSTRY]->(Industry)

The practical consequence of this structure is that queries which would require multi-way joins in SQL become short Cypher path patterns. Finding all articles that connect a given author to a given government body is a two-hop traversal: (Author)-[:WROTE]->(Article)-[:MENTIONS_GOVT_BODY]->(GovtBody). Finding which companies co-occur most frequently with a given market sector is three hops: (Company)<-[:MENTIONS_COMPANY]-(Article)-[:TAGGED_SECTOR]->(Sector). These patterns execute via index-free adjacency without any join resolution, and their performance scales with the size of the local neighbourhood rather than the size of the full dataset.

Beyond query efficiency, the graph model also makes the dataset's inherent connectivity directly visible. Articles sharing the same GV government body and RE region are latently related in a way that is invisible in a flat or document-oriented representation but emerges immediately from graph structure. This is particularly valuable for a news corpus, where the overlap of topics, institutions, and geographies across articles is itself analytically meaningful.

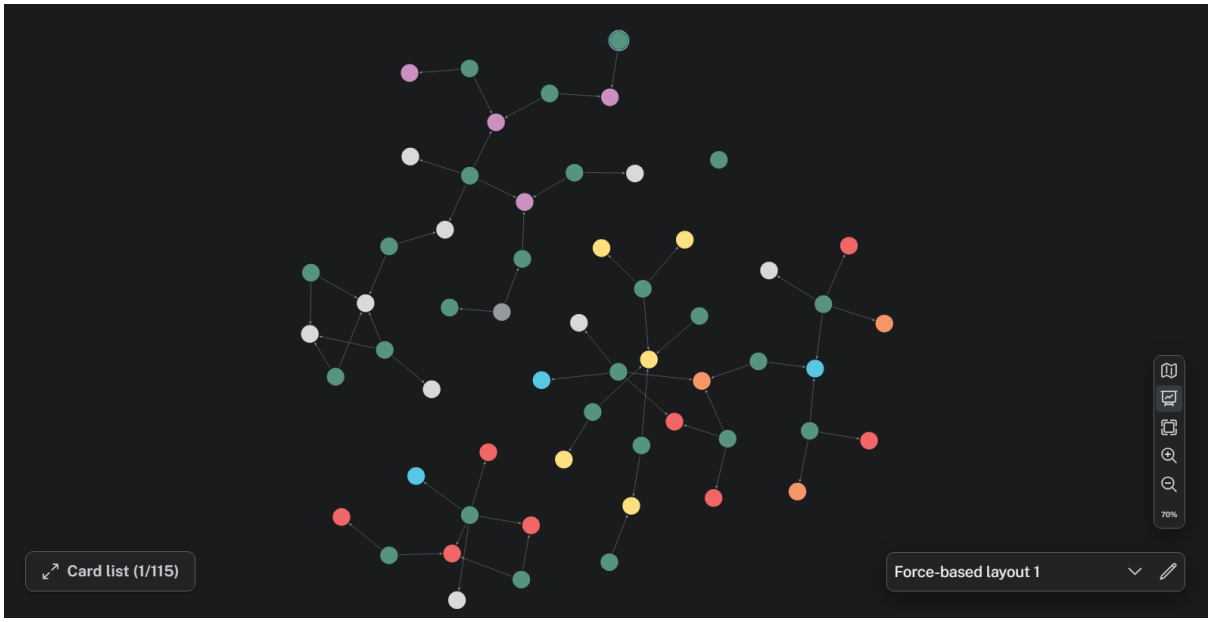


Nodes, Relationships & Properties
(Designing, building, and populating the database)

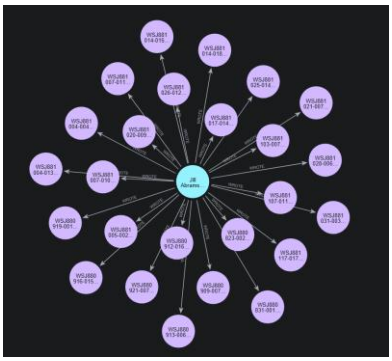
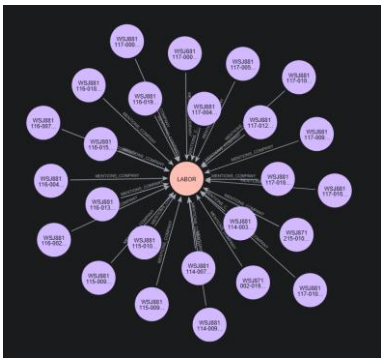
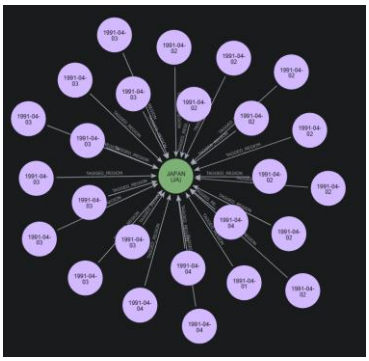


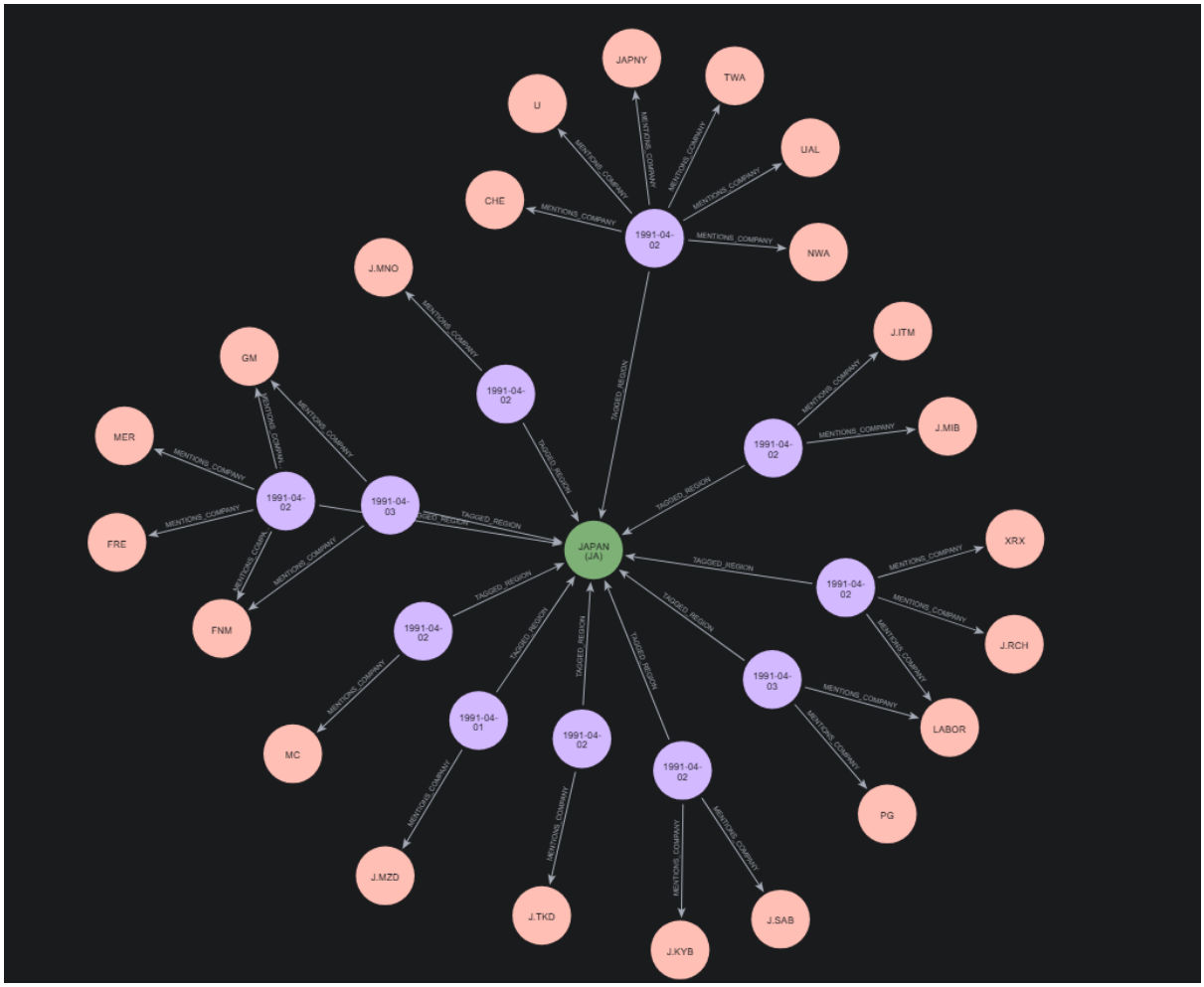
Interdependent Relationships

3.3 Graph Analysis Use Cases

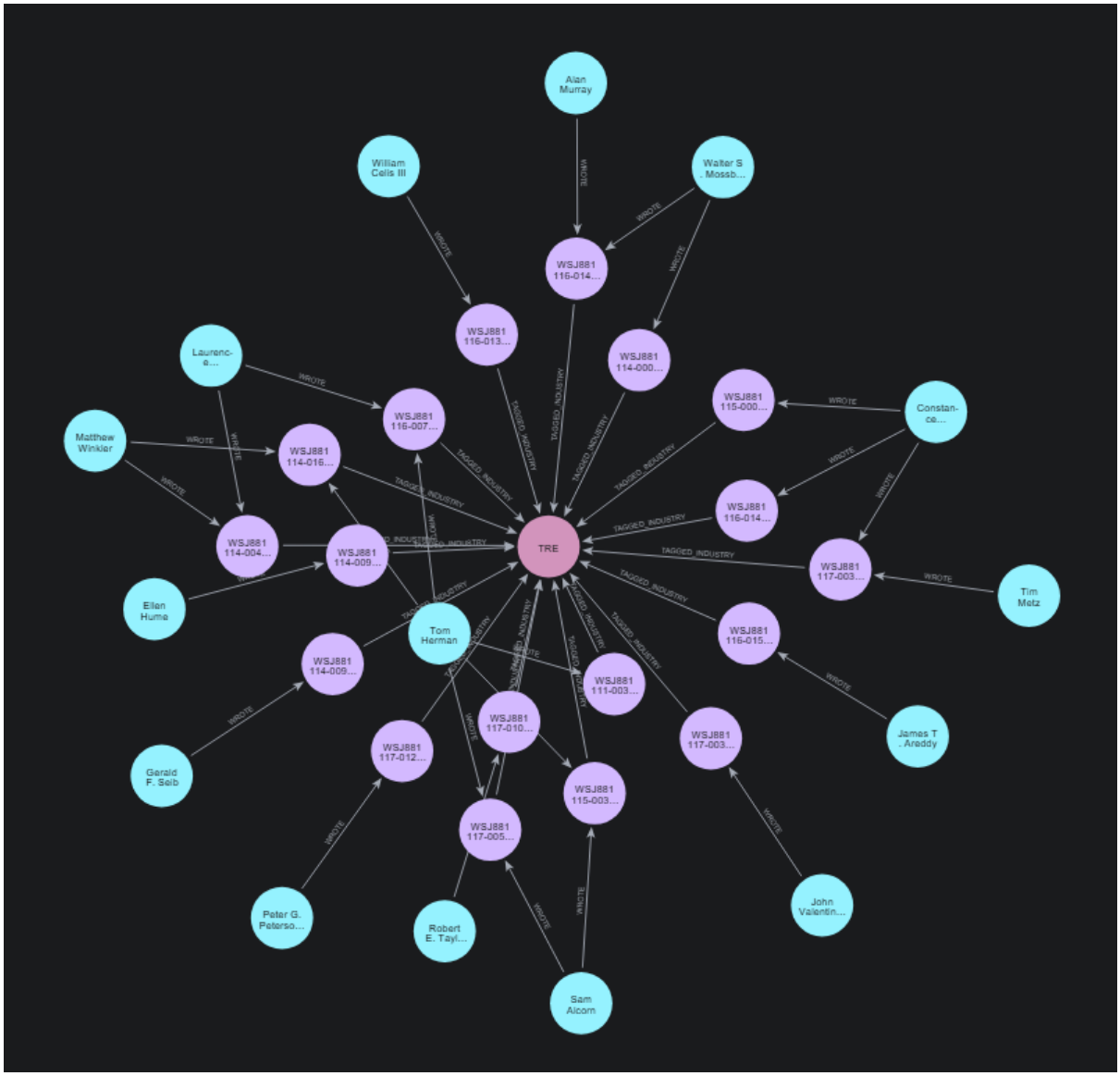


WSJ Graph Model (Actual sample screenshot)

		
Author nodes, Article nodes, and the wrote relationships between them	*Article nodes, Company nodes, and the mentioned company relationships*	*Article nodes, Region nodes, and the tagged region relationships*



Which companies are most frequently mentioned in articles tagged to a given geographic region?
(Company)<-[:MENTIONS_COMPANY]-(Article)-[:TAGGED_REGION]->(Region)

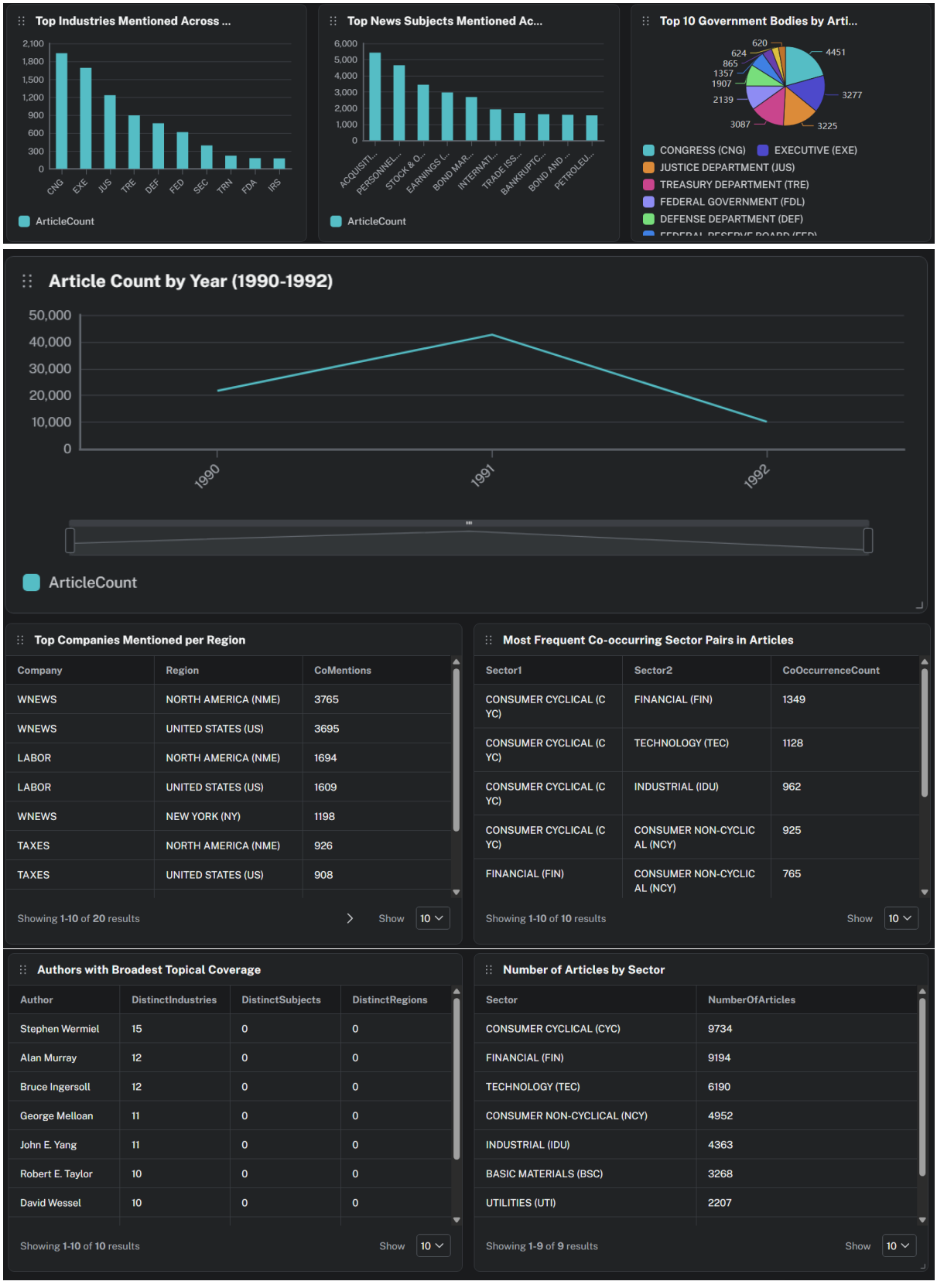


Authors by broadest industry coverage

Other interesting applications cases are listed below :

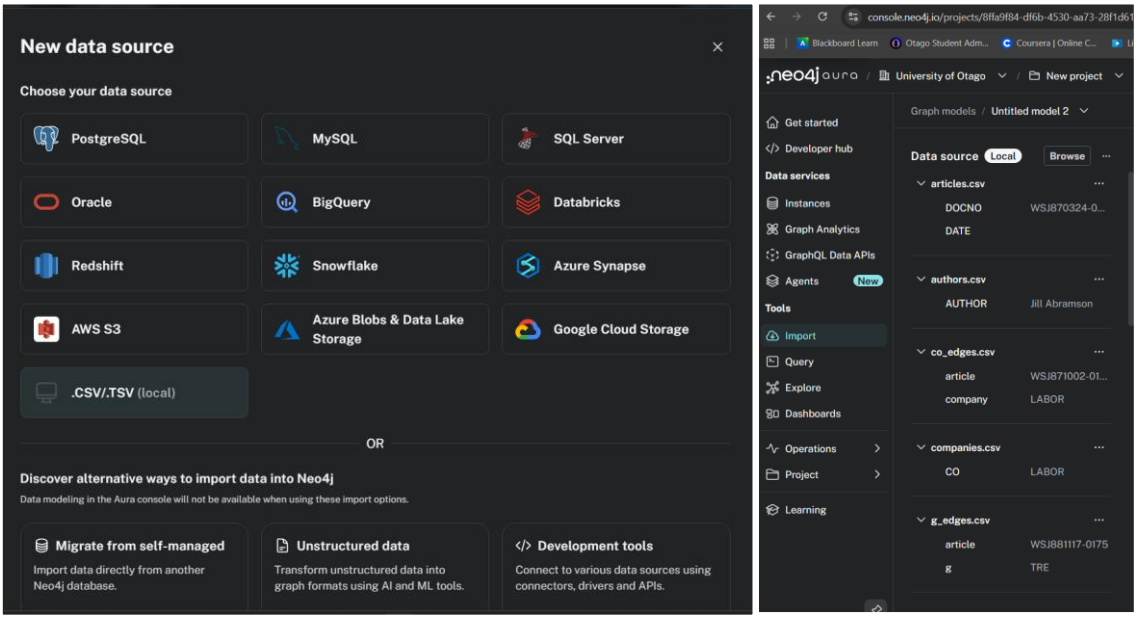
- Author–topic profiling : which industry sectors (IN) or news subjects (NS) does a given author write about, and how does coverage evolve across the 1990–1992 window? This is a two-hop traversal through :WROTE and :TAGGED_INDUSTRY with date filtering on Article properties.
- Government body coverage : which government bodies (GV) receive the most article coverage, and which authors drive that coverage? A three-hop pattern: (Author)-[:WROTE]->(Article)-[:MENTIONS_GOVT_BODY]->(GovtBody).
- Company–region co-occurrence : which companies are most frequently mentioned in articles tagged to a given geographic region? (Company)<-[:MENTIONS_COMPANY]-(Article)-[:TAGGED_REGION]->(Region).
- Thematic clustering : which pairs of market sectors (MS) most frequently appear together across articles? A two-hop co-occurrence: (Sector)<-[:TAGGED_SECTOR]-(Article)-[:TAGGED_SECTOR]->(Sector).
- Editorial footprint : which authors have the broadest topical coverage, measured by distinct IN, NS, and RE values reached through their articles?

3.4 Relational Dashboard



4 Appendix - Data Import

4.1 Import Method and Tools



Used inbuilt capabilities to import data using a Cypher Script (Refer Script Below)

4.2 Schema Code & Data Import Script

Nodes Script

Subject

Key statement

```
CREATE CONSTRAINT `subjectIdentifier_Subject_key` IF NOT EXISTS
FOR (n: `Subject`)
REQUIRE (n.`subjectIdentifier`) IS NODE KEY;
```

Load statement

```
UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`NS` IN $idsToSkip AND NOT nodeRecord.`NS` IS NULL
MERGE (n: `Subject` { `subjectIdentifier`: nodeRecord.`NS` });
```

Sector

Key statement

```
CREATE CONSTRAINT `sectorIdentifier_Sector_key` IF NOT EXISTS
FOR (n: `Sector`)
REQUIRE (n.`sectorIdentifier`) IS NODE KEY;
```

Load statement

```
UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`MS` IN $idsToSkip AND NOT nodeRecord.`MS` IS NULL
MERGE (n: `Sector` { `sectorIdentifier`: nodeRecord.`MS` });
```

Region

Key statement

Edges Script

TAGGED_SUBJECT

Load statement

```
UNWIND $relRecords AS relRecord
MATCH (source: `Article` { `documentNumber`: relRecord.`article` })
MATCH (target: `Subject` { `subjectIdentifier`: relRecord.`subject` })
MERGE (source)-[r: `TAGGED_SUBJECT`]->(target);
```

TAGGED_SECTOR

Load statement

```
UNWIND $relRecords AS relRecord
MATCH (source: `Article` { `documentNumber`: relRecord.`article` })
MATCH (target: `Sector` { `sectorIdentifier`: relRecord.`sector` })
MERGE (source)-[r: `TAGGED_SECTOR`]->(target);
```

TAGGED_REGION

Load statement

```
UNWIND $relRecords AS relRecord
MATCH (source: `Article` { `documentNumber`: relRecord.`article` })
MATCH (target: `Region` { `regionIdentifier`: relRecord.`region` })
MERGE (source)-[r: `TAGGED_REGION`]->(target);
```

MENTIONS_GOVT_BODY

```
CREATE CONSTRAINT `regionIdentifier_Region_key` IF NOT EXISTS
FOR (n: `Region`)
REQUIRE (n.`regionIdentifier`) IS NODE KEY;
```

Load statement

```
UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`RE` IN $idsToSkip AND NOT nodeRecord.`RE` IS NULL
MERGE (n: `Region` { `regionIdentifier`: nodeRecord.`RE` });
```

GovtBody

Key statement

```
CREATE CONSTRAINT `govtBodyName_GovtBody_key` IF NOT EXISTS
FOR (n: `GovtBody`)
REQUIRE (n.`govtBodyName`) IS NODE KEY;
```

Load statement

```
UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`govt_body` IN $idsToSkip AND NOT
nodeRecord.`govt_body` IS NULL
MERGE (n: `GovtBody` { `govtBodyName`: nodeRecord.`govt_body` });
```

Industry

Key statement

```
CREATE CONSTRAINT `gNodeIdentifier_Industry_key` IF NOT EXISTS
FOR (n: `Industry`)
REQUIRE (n.`gNodeIdentifier`) IS NODE KEY;
```

Load statement

```
UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`G` IN $idsToSkip AND NOT nodeRecord.`G` IS NULL
MERGE (n: `Industry` { `gNodeIdentifier`: nodeRecord.`G` });
```

Article

Key statement

```
CREATE CONSTRAINT `documentNumber_Article_key` IF NOT EXISTS
FOR (n: `Article`)
REQUIRE (n.`documentNumber`) IS NODE KEY;
```

Load statement

```
UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`DOCNO` IN $idsToSkip AND NOT nodeRecord.`DOCNO`
IS NULL
MERGE (n: `Article` { `documentNumber`: nodeRecord.`DOCNO` })
SET n.`date` = nodeRecord.`DATE`;
```

Author

Key statement

```
CREATE CONSTRAINT `authorName_Author_key` IF NOT EXISTS
FOR (n: `Author`)
REQUIRE (n.`authorName`) IS NODE KEY;
```

Load statement

```
UNWIND $nodeRecords AS nodeRecord
WITH *
```

Load statement

```
UNWIND $relRecords AS relRecord
MATCH (source: `Article` { `documentNumber`: relRecord.`article` })
MATCH (target: `GovtBody` { `govtBodyName`: relRecord.`govt_body` })
MERGE (source)-[r: `MENTIONS_GOVT_BODY`]->(target);
```

TAGGED_INDUSTRY

Load statement

```
UNWIND $relRecords AS relRecord
MATCH (source: `Article` { `documentNumber`: relRecord.`article` })
MATCH (target: `Industry` { `gNodeIdentifier`: relRecord.`g` })
MERGE (source)-[r: `TAGGED_INDUSTRY`]->(target);
```

MENTIONS_COMPANY

Load statement

```
UNWIND $relRecords AS relRecord
MATCH (source: `Article` { `documentNumber`: relRecord.`article` })
MATCH (target: `Company` { `companyIdentifier`: relRecord.`company` })
MERGE (source)-[r: `MENTIONS_COMPANY`]->(target);
```

WROTE

Load statement

```
UNWIND $relRecords AS relRecord
MATCH (source: `Author` { `authorName`: relRecord.`author` })
MATCH (target: `Article` { `documentNumber`: relRecord.`article` })
MERGE (source)-[r: `WROTE`]->(target);
```

```
WHERE NOT nodeRecord.`AUTHOR` IN $idsToSkip AND NOT
nodeRecord.`AUTHOR` IS NULL
MERGE (n: `Author` { `authorName`: nodeRecord.`AUTHOR` });
```

Company

Key statement

```
CREATE CONSTRAINT `companyIdentifier_Company_key` IF NOT EXISTS
FOR (n: `Company`)
REQUIRE (n.`companyIdentifier`) IS NODE KEY;
```

Load statement

```
UNWIND $nodeRecords AS nodeRecord
WITH *
WHERE NOT nodeRecord.`CO` IN $idsToSkip AND NOT nodeRecord.`CO` IS NULL
MERGE (n: `Company` { `companyIdentifier`: nodeRecord.`CO` });
```

4.3 Data Preprocessing Script

```
library(stringr)
library(purrr)
library(dplyr)
library(tibble)
library(xml2)
library(tidyr)
setwd("C:/Users/siddh/Downloads/DATA404/Assignment 2")

########################################################################
# Load, Clean & Prepare Raw Data
########################################################################
# Read WSJ XML safely
wsjxml <- readLines("wsj.xml", warn = FALSE)
# collapse lines into one string
wsjtxt <- paste(wsjxml, collapse = "\n")
# Fix broken ampersands
wsjtxt <- gsub("&(?!amp;|lt;|gt;|quot;|apos;)", "&amp;", wsjtxt, perl = TRUE)
# Remove illegal control characters
wsjtxt <- gsub("[[:cntrl:]]", "", wsjtxt)
# Wrap in root node
wsjtxt <- paste0("<ROOT>\n", wsjtxt, "\n</ROOT>")

########################################################################
# Sanity Checks | Extract all tags
########################################################################
tags_regex <- gregexpr("<\\/?[A-Za-z0-9_:-]+", wsjtxt, perl = TRUE)
raw_tags <- regmatches(wsjtxt, tags_regex)[[1]]
tags <- unique(gsub("<\\/?", "", raw_tags))
rm(tags_regex, raw_tags)

########################################################################
# Sanity Checks | Check tag mismatches
########################################################################
check_tag_mismatches <- function(wsjxml, tags) {

  doc_count <- sum(grepl("<DOC>", wsjxml))
```

```
  results <- lapply(tags, function(tag) {

    open_pat  <- paste0("<", tag, "(\\s|>|/)")
    close_pat <- paste0("</", tag, ">")

    open_count  <- sum(grepl(open_pat, wsjxml, perl = TRUE))
    close_count <- sum(grepl(close_pat, wsjxml, perl = TRUE))

    tibble(
      tag = tag,
      open_count = open_count,
      close_count = close_count,
      doc_count = doc_count,
      count_equal = open_count == close_count,
      count_within_range = (open_count+close_count)/2 <= doc_count,
    )
  })

  bind_rows(results)
}

# Run mismatch check
mismatch_df <- check_tag_mismatches(wsjxml, tags)
mismatch_df

######################################################################
# Parse XML
######################################################################
doc <- read_xml(wsjtxt)
docs <- xml_find_all(doc, ".//DOC")
df <- data.frame(
  DOCNO    = xml_text(xml_find_first(docs, ".//DOCNO")),
  DOCID    = xml_text(xml_find_first(docs, ".//DOCID")),
  DATE     = xml_text(xml_find_first(docs, ".//DATE")),
  AUTHOR   = xml_text(xml_find_first(docs, ".//AUTHOR")),
  DATELINE = xml_text(xml_find_first(docs, ".//DATELINE")),
  AN       = xml_text(xml_find_first(docs, ".//AN")),
  CO       = xml_text(xml_find_first(docs, ".//CO")),
  DD       = xml_text(xml_find_first(docs, ".//DD")),
  G        = xml_text(xml_find_first(docs, ".//G")),
  GV       = xml_text(xml_find_first(docs, ".//GV")),
  HL       = xml_text(xml_find_first(docs, ".//HL")),
  IN       = xml_text(xml_find_first(docs, ".//IN")),
  LP       = xml_text(xml_find_first(docs, ".//LP")),
  MS       = xml_text(xml_find_first(docs, ".//MS")),
  NS       = xml_text(xml_find_first(docs, ".//NS")),
  RE       = xml_text(xml_find_first(docs, ".//RE")),
  SO       = xml_text(xml_find_first(docs, ".//SO")),
  ST       = xml_text(xml_find_first(docs, ".//ST")),
  TEXT     = xml_text(xml_find_first(docs, ".//TEXT")),
  stringsAsFactors = FALSE
)

######################################################################
# Clean & Prepare Parsed Data
######################################################################
```

```
# 1. Trim whitespace in all columns
df <- df %>%
  mutate(across(everything(), ~ str_trim(as.character(.))))

# 2. Remove trailing "." from DOCID & AN
df <- df %>%
  mutate(DOCID = str_remove(DOCID, "\\.$"))
# 2. Remove trailing "." from DOCID
df <- df %>%
  mutate(AN = str_remove(AN, "\\.$"))

# 3. Fill missing DOCID using DOCNO
df <- df %>%
  mutate(
    DOCID = ifelse(
      is.na(DOCID) | DOCID == "",
      str_remove(DOCNO, "^WSJ"),
      DOCID
    )
  )

# 4. Convert DATE to Date format (MM/DD/YY)
df <- df %>%
  mutate(DATE = as.Date(DATE, format = "%m/%d/%y"))

########################################################################
# FUNCTION: Parse DD column (mixed formats)
########################################################################
parse_dd <- function(x) {
  x <- str_trim(as.character(x))

  if (grepl("^\\d{6}$", x)) {
    return(as.Date(x, format = "%y%m%d"))
  } else if (grepl("/", x)) {
    return(as.Date(x, format = "%m/%d/%y"))
  } else {
    return(NA)
  }
}

df <- df %>%
  mutate(DD2 = as.Date(sapply(DD, parse_dd), origin = "1970-01-01"))
########################################################################
df <- df %>%
  mutate(
    AUTHOR = str_replace(
      AUTHOR,
      ", Staff Reporter of The Wall [Ss]treet (Journal|Jornal)",
      " (WSJ Staff)"
    )
  )
df <- df %>%
  mutate(WSJ_Staff = ifelse(str_detect(AUTHOR, "\\(WSJ Staff\\)"), 1, 0))
df <- df %>%
  mutate(WSJ_Special = ifelse(str_detect(AUTHOR, "\\(WSJ Special\\)"), 1, 0))
```

```
df <- df %>%
  mutate(
    AUTHOR = str_trim(
      str_remove(
        AUTHOR,
        "\\s*\\((WSJ Staff|WSJ Special)\\)\\s*$"
      )
    )
  )
######################################################################
# CHECK RESULT
######################################################################
# AUTHOR — & separated
df_authors <- df %>%
  dplyr::select(DOCNO, AUTHOR) %>%

  mutate(AUTHOR = ifelse(is.na(AUTHOR), "", AUTHOR)) %>%
  mutate(AUTHOR = stringr::str_replace_all(AUTHOR, "\\band\\b", ",")) %>%
  mutate(AUTHOR = stringr::str_replace_all(AUTHOR, "&", ",")) %>%

  tidyr::separate_rows(AUTHOR, sep = ",") %>%

  mutate(AUTHOR = stringr::str_squish(AUTHOR)) %>%

  filter(AUTHOR != "")

# CO — space-separated
df_co <- df %>%
  dplyr::select(DOCNO, CO) %>%
  mutate(CO = ifelse(is.na(CO), "", CO)) %>%
  tidyr::separate_rows(CO, sep = "\\s+") %>%
  mutate(CO = stringr::str_squish(CO)) %>%
  filter(CO != "")

# G — space-separated
df_g <- df %>%
  dplyr::select(DOCNO, G) %>%
  mutate(G = ifelse(is.na(G), "", G)) %>%
  tidyr::separate_rows(G, sep = "\\s+") %>%
  mutate(G = stringr::str_squish(G)) %>%
  filter(G != "")

# GV — closing parenthesis separated
df_gv <- df %>%
  dplyr::select(DOCNO, GV) %>%
  mutate(GV = ifelse(is.na(GV), "", GV)) %>%
  tidyr::separate_rows(GV, sep = "\\)") %>%
  mutate(GV = stringr::str_squish(GV)) %>%
  filter(GV != "") %>%
  mutate(GV = paste0(GV, ")"))

# RE — closing parenthesis separated
df_re <- df %>%
  dplyr::select(DOCNO, RE) %>%
  mutate(RE = ifelse(is.na(RE), "", RE)) %>%
```

```
  tidyr::separate_rows(RE, sep = "\\)") %>%
  mutate(RE = stringr::str_squish(RE)) %>%
  filter(RE != "") %>%
  mutate(RE = paste0(RE, ")"))

# MS — closing parenthesis separated
df_ms <- df %>%
  dplyr::select(DOCNO, MS) %>%
  mutate(MS = ifelse(is.na(MS), "", MS)) %>%
  tidyr::separate_rows(MS, sep = "\\)") %>%
  mutate(MS = stringr::str_squish(MS)) %>%
  filter(MS != "") %>%
  mutate(MS = paste0(MS, ")"))

# NS — closing parenthesis separated
df_ns <- df %>%
  dplyr::select(DOCNO, NS) %>%
  mutate(NS = ifelse(is.na(NS), "", NS)) %>%
  tidyr::separate_rows(NS, sep = "\\)") %>%
  mutate(NS = stringr::str_squish(NS)) %>%
  filter(NS != "") %>%
  mutate(NS = paste0(NS, ")"))

######################################################################
# WRITE RESULT
######################################################################
# Articles
articles <- df %>%
  dplyr::select(DOCNO, DATE)
write.csv(articles, "articles.csv", row.names = FALSE, na = "")

# Authors
authors <- df_authors %>%
  distinct(AUTHOR)
write.csv(authors, "authors.csv", row.names = FALSE, na = "")

wrote_edges <- df_authors %>%
  rename(article = DOCNO, author = AUTHOR)
write.csv(wrote_edges, "wrote_edges.csv", row.names = FALSE, na = "")

# CO — company identifiers
companies <- df_co %>%
  distinct(CO)
write.csv(companies, "companies.csv", row.names = FALSE, na = "")

co_edges <- df_co %>%
  rename(article = DOCNO, company = CO)
write.csv(co_edges, "co_edges.csv", row.names = FALSE, na = "")

# G
g_nodes <- df_g %>%
  distinct(G)
write.csv(g_nodes, "g_nodes.csv", row.names = FALSE, na = "")

g_edges <- df_g %>%
  rename(article = DOCNO, g = G)
```

```
write.csv(g_edges, "g_edges.csv", row.names = FALSE, na = "")

# GV — government bodies
govt_bodies <- df_gv %>%
  distinct(GV) %>%
  rename(govt_body = GV)
write.csv(govt_bodies, "govt_bodies.csv", row.names = FALSE, na = "")

gv_edges <- df_gv %>%
  rename(article = DOCNO, govt_body = GV)
write.csv(gv_edges, "gv_edges.csv", row.names = FALSE, na = "")

# RE — geographic regions
regions <- df_re %>%
  distinct(RE)
write.csv(regions, "regions.csv", row.names = FALSE, na = "")

re_edges <- df_re %>%
  rename(article = DOCNO, region = RE)
write.csv(re_edges, "re_edges.csv", row.names = FALSE, na = "")

# MS — market sectors
sectors <- df_ms %>%
  distinct(MS)
write.csv(sectors, "sectors.csv", row.names = FALSE, na = "")

ms_edges <- df_ms %>%
  rename(article = DOCNO, sector = MS)
write.csv(ms_edges, "ms_edges.csv", row.names = FALSE, na = "")

# NS — news subjects
subjects <- df_ns %>%
  distinct(NS)
write.csv(subjects, "subjects.csv", row.names = FALSE, na = "")

ns_edges <- df_ns %>%
  rename(article = DOCNO, subject = NS)
write.csv(ns_edges, "ns_edges.csv", row.names = FALSE, na = "")
```